Building a Futures Sim: Lessons from the NFL Model

Oct 9, 2026 · @Corey

The mental model

A futures sim is a counting machine: feed it one strength number per team and a schedule, play the whole season many thousands of times with dice, and every futures price is just "how many of those seasons did X happen, divided by the total." Title odds, top-4, relegation, team points totals, head-to-heads all fall out of the same pile of simulated seasons. Nothing is modeled directly as a market; markets are counted after the fact.

That means the whole quality of the sim lives in three places, in this order of importance:

1. **The ratings.** One number per team that means "expected margin against an average team on a neutral field." If these are wrong, nothing downstream saves you.
2. **The single-match model.** How a rating gap plus home advantage turns into one random result. This must be calibrated to reality (real spreads, real win rates), not guessed.
3. **The uncertainty layers.** A point rating is a lie; the sim has to admit it might be wrong about each team (rating uncertainty) and that teams change during a season (drift). These two layers are what make the tails right, and the tails are where futures prices live.

Everything else (tiebreakers, injuries, rest, cut-off weeks) is plumbing that matters at the margins. Get the three above right first.

A design rule worth copying from day one: **every knob lives in one config file, and every input is a plain CSV a human can open and edit.** The NFL sim's config is a single flat object; the dashboard just edits it. Ratings, schedule, results, injuries are all CSVs. The sim is rerun from scratch each time (no incremental state), which keeps it debuggable and makes "what changed since last week" a clean diff of inputs.

Team ratings

A rating is **expected margin against a league-average team at a neutral site**, in the sport's scoring unit (points in the NFL). The sim never uses a rating alone; it only ever uses the gap between two of them plus home advantage. That one definition is what lets ratings from completely different sources be averaged.

Several sources, one blend. The NFL sim keeps a small table of rating sources, each a column of one number per team, with an on/off switch and a weight. The blend step is:

1. Anchor every source so its mean is zero (subtract the column mean). Sources disagree about where "average" is; this removes that argument.
2. Any source not on the points scale (an Elo-style number, a 0-100 power index) is rescaled so its spread matches the typical spread of the point-scale sources. Spread is measured as the standard deviation across teams.
3. Weighted average, then anchor to zero again.

The sources that earn weight are a mix of human judgment and market-derived numbers:

- **Hand ratings** from people whose opinion he trusts (his own, a friend's). These carry the most weight because they encode things the market has not priced yet.
- **A public predictive model's ratings** as a sanity anchor.
- **Market-implied ratings from point spreads.** Every posted spread is one equation: spread = rating(home) minus rating(away) plus home edge. Stack a few hundred of them, add a "ratings sum to zero" row, and solve by least squares. This is the single most useful source to build first, because it is free, objective and recomputable every week.
- **Market-implied ratings from season win totals.** Devig each team's win-total market into an expected wins number, then search for the rating vector whose expected wins over the real schedule reproduce those numbers. Books shade the overs, so the totals are re-centered before solving; only the relative shape carries information.

Two conventions worth stealing.

- **A rating assumes the team's designated starters are healthy.** Short absences are never baked into the rating; they are applied per game as a temporary penalty (next section). A season-ending loss shifts who the "designated" starter is, and the sources are assumed to have already priced that in.

- **Human overrides sit on top of the blend, not inside it.** There is a permanent per-team adjustment column (his judgment, in points) and a separate per-team-per-week temporary adjustment for short-term news. Both are plain CSV columns. The blend itself is never hand-edited, so it can always be regenerated.

The spread of the blended ratings across teams is a number to watch. If the blend is flatter than the market's implied spread, the sim will quietly like the worst teams and dislike the best ones. Compare the two every time the blend changes.

Game randomness

One match is simulated in two steps: build the expected margin, then roll dice around it.

Expected margin. Start with rating(home) minus rating(away) plus a home edge. Then add every per-game adjustment the sim knows about for that specific fixture:

- Home edge is per team (a few tiers), but the tiers are scaled so the league average matches what the market actually prices for home teams. Neutral-site games get their own smaller edge.
- A missing key player costs his team his value in points for that game (the NFL version is a quarterback depth chart with an injury rate per week and a distribution of how long absences last, so injuries happen inside the sim too, not just from the news).
- A rest bump for a team coming off its bye week.
- Late-season situational effects: a team that has clinched plays a little worse (resting), a team with nothing to play for and a locked seed plays a lot worse (starters benched). These are per-game only and never persist.
- Any temporary adjustment the human typed in for that week.

The rule that makes all of this safe: **whatever goes into the expected margin also goes into the expectation used later for drift.** If the sim expects a team to be weaker this week because of an injury, it must not then be "surprised" by a loss. More on that under drift.

The dice. Margin = Normal(expected margin, margin_sd), rounded to a whole number. A rounded zero becomes a draw with a small probability, otherwise an overtime coin flip. The margin SD is not a free parameter; it is measured from history as the standard deviation of (actual margin minus closing spread). Measure it for your league.

The decoupled win curve. This is the subtle one. In the NFL the normal model with the measured margin SD gets margins right but gets win rates wrong: favorites win more often than that bell curve implies. One SD cannot do both jobs. The fix: keep margin_sd for the spread of margins, but sample around a stretched mean, lam × expected margin, where lam = margin_sd / win_sd and win_sd is a second, smaller SD fitted to historical win rates by spread bucket. Favorites then win at the correct steeper rate while margins keep their correct spread. Backtested, this improved both log-loss and tail calibration. Check whether your sport has the same mismatch before copying it; soccer may not, because its scoring process is so different (see the Premier League section).

Win-probability overrides. Any game can be given a win probability directly instead of a rating gap. Internally that becomes an expected margin through the inverse normal: mu = margin_sd × ppf(p). Two uses:

- Manual: when he knows something the ratings do not.
- Automatic for the current round: unplayed games this week are priced off the sportsbook market (devig each book's moneyline, take the median across books, fall back to a consensus line, ignore anything stale). Every later week stays on ratings. A sniff test flags any game where market and ratings disagree by a lot.

Locked results. Completed real games get their real score in every simulated season. The sim is always a full-season rerun from the preseason file with results pinned, never an incremental update.

Rating uncertainty

The rating is a best guess, and the sim has to admit it could be wrong. **At the start of every simulated season, each team's true strength is drawn once: rating plus Normal(0, rating_sd).** That draw is held for the whole season.

The word "once" is the entire point. Per-game dice already give you noise, but it is independent from game to game and averages out over a season. A single season-long draw is correlated across all of a team's games: in some simulated seasons the team is simply better than you thought for 38 matches in a row. That is what produces the fat tails that futures markets price (a mid-table team winning the title, a favorite finishing 6th), and a sim without it is measurably overconfident.

How to size it. Not by feel. The NFL value was set by a backtest: simulate each past season from market-implied preseason ratings, score the predicted season win totals against what happened, and look at two things:

- Log-loss of the win-total distribution.
- Tail calibration: the share of real outcomes that landed in the sim's 10% tails on each side. If 20% of outcomes land in the combined 20% tails the sim is honest; well above that means too confident, below means too wide.

The backtest surface is a ridge, not a point: rating uncertainty and in-season drift (next section) both add variance, and what matters is their total. Several pairs calibrate equally well. Pick a pair on the ridge, then do not touch it mid-season.

Per-team volatility. Some teams genuinely deserve a wider draw: a rookie starter at the key position, a new manager, a squad with huge turnover. The NFL sim has a five-word ladder (way less, less, normal, more, very) stored as a plain word per team. Each word is a multiplier on the global SD, with the rungs spaced evenly in variance rather than in SD, because uncertainty compounds by the square. Two lessons from building it:

- Use it rarely. One to three "very" teams a season, a handful of "more". The backtest evidence for wider rookie outcomes was suggestive, not decisive, so the top rung was deliberately set well below what the data hinted.
- **Means matter more than variance.** The same backtest found the market historically overrates rookie-starter teams by roughly a win. A small negative adjustment to the rating does more than any amount of extra variance.

Does it shrink during the season? No, and this was tested. Restarting past seasons at various mid-season points, the best-calibrating rating_sd did not fall as games were played; if anything it rose. The reason: drift variance already self-decays because it scales with weeks remaining, while the "we are wrong about current strength" share does not shrink, and late-season unmodeled reality (rested starters, attrition) adds noise. Keep one value all season.

Drift inside a simulated season

Teams are not the same in May as in August. Inside every simulated season, ratings move after every simulated game, scaled by how surprising the result was:

```
\text{surprise} = \text{actual margin} - \text{expected margin}
```

```
\text{shift} = \operatorname{clip}(k \times \text{surprise},\; -\text{cap},\; +\text{cap})
```

The winning side gains the shift, the losing side loses it, so the league stays zero-sum. A big underdog that wins by a little moves a lot; a favorite that wins by exactly what was expected moves nothing.

The expected margin in that formula is the sim's own belief for that game, including home edge, injuries, rest and any override, and including the stretched mean from the decoupled win curve. Because the dice were rolled around that same expectation, the average surprise is exactly zero and drift adds variance without adding bias. Break this rule (say, by computing surprise against the raw rating gap while the dice used an injury-adjusted one) and every injured team drifts upward for no reason.

The cap matters. Uncapped drift gave slightly better log-loss in backtests but worse tails: one freak blowout should not rewrite a team. Capped at a modest fraction of a typical rating gap it calibrated best.

The shape was tested against alternatives. Constant k beat a Kalman-style shrinking gain, a decaying k, and a tanh-compressed surprise. The simplest rule won; only its value needed fitting.

Two k values, not one. Simulated games use k_sim. Real completed games, when replayed into the ratings, use a separate k_real, and his k_real is zero. The reasoning: the rating sources already incorporate real results (the market-implied ratings are refit every week), so applying drift to real results on top would double count. "Our number might be wrong" belongs to rating uncertainty, not to drift. He raises k_real only as a deliberate in-season judgment when the sources are stale. The backtest found a small positive k_real still had value even with weekly refreshed sources, so the honest range is zero to small.

The eliminated-team dock. The moment a team is mathematically out of the playoffs inside a simulated season, its rating drops once and stays down for the rest of that season. Markets discount dead teams and real dead teams play like it. The dock only applies to teams whose playoff odds entering the run were meaningful; a team already priced as hopeless is assumed to carry the discount in its rating, so it is not docked twice.

What drift does to the numbers. Comparing against a commercial sim run on identical ratings, drift plus rating uncertainty together shaved a couple of percentage points off the top team's title probability and spread it across the field. That is deliberate and backtest-supported. A sim with static ratings prices the favorite too short.

How ratings change week to week in the real world

Drift is what happens inside a simulated season. This section is what the human does between real rounds. The NFL weekly routine, in order:

1. **Results in.** The real scores for the round go into the results file. From now on those games are locked in every simulation.
2. **Sources refresh.** The market-implied ratings are refit from the newest closing lines (causally: only lines through the round just played). Hand ratings are re-entered when their authors update them. Scraped public ratings are re-pulled. The blend is recomputed, not edited.
3. **Availability updated.** Who is out, for how long, and past weeks marked as "was out" so the history stays honest.
4. **Temporary adjustments** for this week's news that no source has priced yet.
5. **The current round is priced off the market**, every later round off ratings.
6. **Full rerun** from scratch. Every run archives its complete inputs, so any past run can be reopened exactly as it was.

Keep a rating history. One row per team per week of the blended rating. Plotting it is the fastest way to notice a source going stale, a team the market has moved on that the hand ratings have not, or a blend that drifted flat. The NFL sim also keeps the weekly fair odds for each futures market, so he can see how his own prices moved.

The one rule. The sources are trusted to incorporate results. Real results are not fed back into ratings through the drift formula on top of that (k_real is zero) because it double counts. If a source stops updating, lower its weight; do not bolt on a correction.

How much do teams really move? A lot more than beginners assume. In the NFL, a meaningful fraction of the league ends the season several points away from where the preseason market had them, and the spread of that movement is larger than the typical rating gap between adjacent teams. Any weekly process that keeps ratings near their preseason values is wrong. Let the market-implied source move freely and weight it well.

The season loop and counting markets

The engine is a few big arrays, not a loop over seasons. With S simulated seasons and G fixtures, results are an S × G matrix and ratings are an S × teams matrix that gets updated round by round. Every round:

1. Pick that round's fixtures. Compute the expected margin for all of them across all S seasons at once.
2. Roll the dice. Overwrite the columns of games already played in real life with the real score.
3. Apply drift to the ratings matrix. Update running standings.

This order of operations is why a team may only play once per round: the drift update for its first game must land before its second game is priced. Define rounds by match date, not by "week", so midweek fixtures get their own round.

Standings and tiebreakers are computed per simulated season from the results matrix. The NFL has a long tiebreaker cascade; the Premier League's is short (points, goal difference, goals for, head-to-head). Implement the real one exactly, because futures on exact finishing position are decided by tiebreakers more often than people expect. Then play any knockout stage per season.

Counting. Every market is a count over the S seasons divided by S:

- Title, top 4, top 6, relegation: share of seasons the team finished in that band.
- Exact finishing position: a full distribution per team, which also gives "to finish bottom" and "to finish above team X".
- Season points total: the mean is expected points; the over/under at a line is the share of seasons at or above it.
- Head-to-head finishing position: share of seasons A finished above B.

Fair odds are just 1 divided by the probability, converted to whatever odds format the reader uses. Show three decimals on screen, keep full precision in exports.

Sim count. Use a quick setting for iteration and a big one for published numbers. Longshot futures are the reason: an event at one in a thousand needs tens of thousands of simulated seasons before its price stops jumping. The NFL sim runs a million seasons split into chunks across worker processes.

Reproducibility is a hard rule. Same seed plus same inputs gives identical output regardless of how many workers run it. Each chunk gets a seed derived from the master seed and its index. Anything that breaks this (a time-based seed, a shared random generator) is a bug, because without it you cannot tell a real input change from noise.

Calibration and sanity checks

A sim that has never been scored against history is a random number generator with a nice interface. Three layers of checking, cheapest first.

Match-level checks against the last several seasons (an afternoon of work, do it before anything else):

- Home win, draw and away win rates. Your match model must reproduce them.
- Standard deviation of (actual margin minus closing line). This sets the margin noise directly.
- Favorite win rate by closing-line bucket. If favorites win more often than your model says, you have the NFL's mismatch and need either a second SD or a different distribution.
- Home edge in the market versus realized home edge, and whether it has moved recently.
- Rest and congestion effects: results after a short turnaround, after a midweek European match.

Season-level backtest (a day, and the single most valuable thing he can build):

1. For each past season, build preseason market-implied ratings from the opening rounds' lines.
2. Simulate the season from those ratings with a grid of (rating uncertainty, drift k) pairs.
3. Score predicted season points totals against actual by log-loss, and check the tails: the share of real outcomes landing in the sim's 10% tails each side should be about 20% combined. Higher means too confident.
4. Pick a pair on the calibrated ridge and freeze it.

Live comparison against sportsbooks every week:

- Devig futures pools with the power method, not by scaling implied probabilities to sum to one. Simple scaling overstates longshots, and futures are all longshots.
- Only devig complete mutually exclusive pools (the title market, the relegation market). One-sided prices like "top 4 yes" or a points total are compared raw and flagged as such.
- Expected points versus the market's points totals, team by team, after correcting for the books' shade (the market's totals sum to more than the games available).
- A big gap on one team is usually a ratings disagreement; a gap on every top team at once is usually a variance setting.

Engine tests to keep forever. Same seed gives identical output. Turning any feature off reproduces the engine bit for bit as it was before that feature existed. The mean surprise across all simulated games is zero. The NFL sim has a test file run after every engine change, and any change that moves probabilities is flagged in plain words before it ships. Separate "data plumbing" changes from "model logic" changes in your own head and in your notes.

Translating this to the Premier League

Almost everything above carries over unchanged: ratings as expected margin, blending anchored sources, a season-long uncertainty draw, capped drift against the sim's own expectation, full reruns with locked results, power devig, backtest on points totals. What changes is the match model and the league structure.

Goals, not points: use a Poisson model. Football scores are small counts, so a normal margin with rounding is the wrong shape. The standard approach: give each match an expected goals number for each side and draw each side's goals from a Poisson. The draw probability then falls out naturally, so there is no tie hack. Two ways to get the two expected-goals numbers:

- **Simple, recommended for version one.** Keep a single rating per team in goals (expected goal difference versus an average side at a neutral venue). Expected goal difference for a match = rating(home) minus rating(away) plus home edge in goals. Split it around the league's average total goals per match: home mean = half the average total plus half the difference, away mean = half the average total minus half the difference. Clamp both means above a small floor.
- **Fuller.** Separate attack and defence ratings per team in log space (home mean = exp(base + attack_home minus defence_away + home edge), away mean likewise). More parameters, better totals markets, harder to blend with hand ratings. Save for version two.

Independent Poissons slightly under-predict 0-0 and 1-1 and over-predict 1-0 either way. The Dixon-Coles correction is a small multiplier on those four scorelines and is worth adding once the basic model is calibrated. Check the draw rate first; if it already matches the last five seasons, leave it.

The decoupled win curve is probably unnecessary. It fixed a specific normal-model flaw. With Poisson, check favorite win rate by closing-odds bucket instead; if it matches, there is nothing to fix.

Home edge in goals has fallen in recent seasons and was briefly near zero in empty stadiums. Measure it from the last three or four seasons' closing Asian handicap lines, not from older data, and keep it per team only if the per-team numbers are scaled back to the league average.

Rating sources he can build this month:

- Market-implied from closing Asian handicap lines and totals, by least squares. Same equation as the NFL version with goals as the unit. This is the backbone.
- Market-implied from title, top-4 and relegation odds, by solving for the rating vector that reproduces them over the real schedule. Harder, do it later.
- Expected-goals (xG) based strength from public match data, which smooths out finishing luck. A strong, free, objective second source. Convert to goals-vs-average and let the blend rescale it.
- A public Elo-style rating, rescaled to the goals scale by the blend step.
- His own hand ratings, kept on the goals scale from day one.

Uncertainty and drift in goals. The same two knobs, the same backtest; the units are just goals. Thirty-eight games average out per-match noise even more than seventeen do, so the season-long uncertainty draw matters relatively more in football, and the drift k will be a smaller number per goal of surprise. Let the backtest decide.

Where the NFL's extra layers map:

- The quarterback layer has no equivalent. One player is rarely that much of the team. A small "key player out" temporary adjustment column covers it for version one.
- The bye-week bump becomes a **congestion penalty**: a small per-game deduction for a side playing three days after a European or cup match, and perhaps a smaller one for any short turnaround. Measure it before setting it.
- The eliminated-team dock becomes two docks: a team that is mathematically relegated, and a team that is mathematically safe with nothing to play for. Both play worse. Squads chasing Europe or survival play up.
- The week-18 rest logic becomes **final-day dead rubbers**, same idea.
- Per-team volatility goes to promoted sides, new managers, and clubs that turned the squad over in the summer. A mid-season manager sacking is a reason to bump a club's volatility and refresh its rating, not to apply drift.
- The January transfer window is a sources-refresh event. Expect ratings to move more around it.

Structure. Thirty-eight rounds, double round robin, no divisions, no knockout stage, so the standings code is short. Tiebreakers: points, goal difference, goals scored, then head-to-head points and head-to-head away goals, then a playoff that has never happened. Rounds must be ordered by date because of midweek fixtures and postponed games; a postponed match is just a fixture whose round moved. Markets to count: winner, top 4, top 6, relegation, bottom, exact position, points totals, and finish-above head-to-heads.

Current round off the market is easier than in the NFL: devig each book's three-way odds with the power method, take the median across books, and feed the home-win, draw and away-win probabilities straight in. For a Poisson model the override replaces the match's expected goals with the pair that reproduces those probabilities, or more simply draws the result from the three probabilities directly for that round only.

Build order for version one

Each step produces something usable on its own. Do not start the next until the current one is checked.

1. **Files first.** A schedule CSV (date, round, home, away), a results CSV (same keys plus goals), a ratings CSV (team, rating in goals), and one flat config file for every knob. Python with numpy and pandas is enough; no database.
2. **The match model.** Rating gap plus home edge to two Poisson means. Check home, draw and away rates and the scoreline distribution against the last five seasons before writing anything else.
3. **The season loop and counting.** Vectorized over seasons, rounds in date order, real results locked, standings with the real tiebreakers, counts for every market. Aim for a hundred thousand seasons in well under a minute.
4. **Market-implied ratings** from closing Asian handicap lines. Compare the spread of these ratings to the spread of his hand ratings. Build the blend with anchoring and rescaling.
5. **Season-long rating uncertainty**, then the backtest on past seasons' points totals with log-loss and tail checks.
6. **Capped drift** against the sim's own expectation, gridded together with the uncertainty knob. Freeze the pair.
7. **The weekly routine**: results in, sources refreshed, current round off the market, full rerun, rating history row appended, run inputs archived.

8. **Book comparison** with power devig on the complete pools, expected points versus market totals.

Only after all eight: Dixon-Coles, congestion penalty, dead-rubber and relegated-team docks, per-team volatility, attack and defence split.

Traps that cost the NFL sim real time:

- Feeding real results into both the sources and the drift formula. Pick one.
- Computing drift surprise against a different expectation than the dice used. Every injured or rested team then drifts for free.
- Devigging longshots by simple scaling. Use the power method.
- Too few simulated seasons for longshot markets, then chasing noise.
- Changing a calibrated knob mid-season because one team's price looked wrong. That is a ratings problem, not a variance problem.
- Hand-editing the blended output instead of the sources or the adjustment column. The blend must always be regenerable.
- Forgetting to anchor sources to zero before averaging them.
- Looping by week instead of by date, which breaks the one-game-per-round rule on midweek fixtures.
- Treating another sim's output as a calibration target without checking it is internally consistent with the ratings fed to it.
- Shipping a model change without a plain-words note that it moves probabilities. Keep plumbing changes and model changes separate in the log.